Programação Orientada a Objetos

OPERADOR DE ATRIBUIÇÃO

em C++

Introdução

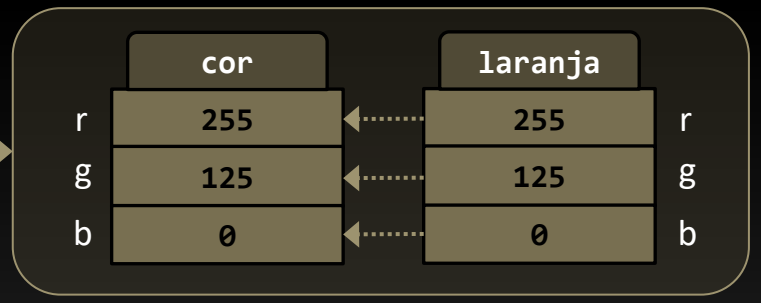
- C++ permite a atribuição entre objetos
  - Igual a atribuição entre registros

```
class Color
{
private:
    int r;  // red
    int g;  // green
    int b;  // blue

public:
    Color();
    Color(int R, int G, int B);
    ~Color();
};
```

```
int main()
{
    Color laranja { 255, 255, 0 };
    Color cor;

    cor = laranja;
}
```



Introdução

- A **atribuição** é feita pelo método operator=
  - Usado em atribuições

```
Color amarelo { 255, 255, 0 };   // construtor
Color cor;                       // construtor padrão
```

```
cor = amarelo;                   // chamada implícita de operator=
cor.operator=(amarelo);          // chamada explícita de operator=
```

  - Não em inicializações

```
Color amarelo { 255, 255, 0 };   // construtor
```

```
Color cor = amarelo;             // construtor de cópia
Color cor = Color(amarelo);      // construtor de cópia explícito
```

Introdução

- Para classes criadas pelo programador
  - O método é definido automaticamente
  - Se nenhum for fornecido

Retorna um objeto
(por referência)

Recebe um objeto
(por referência)

```
Classe & operator=( const Classe & );
```

```
Color & operator=(const Color &);
String & operator=(const String &);
Jogador & operator=(const Jogador &);
```

Atribuição de Objetos

- O método operator= criado automaticamente:
  - Realiza uma cópia "rasa"
    - Similar ao construtor de cópia
    - Copia apenas os atributos

```
Color laranja { 255, 125, 0 };
Color cor;
cor = laranja;

cor.operator=(laranja);
```

```
// criado pelo compilador
Color & operator=(const Color & c)
{
    r = c.r;
    g = c.g;
    b = c.b;
}
```

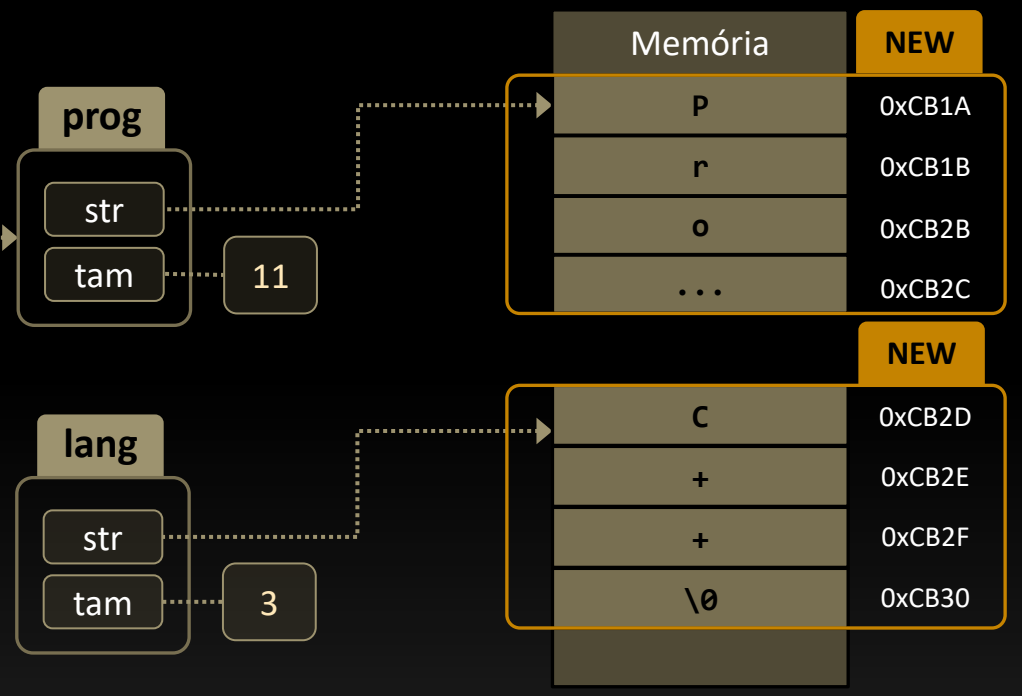
Alocação Dinâmica

- O que acontece se atribuirmos um objeto que utiliza alocação dinâmica de memória

```
int main()
{
    String prog { "Prog" };
    String lang { "C++" };

    prog = lang;
}
```

prog recebe os atributos de lang



Alocação Dinâmica

- Precisamos definir um operador de atribuição
  - Ele deve realizar uma cópia "profunda"

```
int main()
{
    String prog { "Prog" };
    String lang { "C++" };

    prog = lang;
}
```

```
prog.operator=(lang);
```

```
String & String::operator=(const String & s)
{
```

1. Alocar novo bloco de memória
2. Copiar dados para novo bloco
3. Liberar bloco antigo

```
}
```

Operador de Atribuição

- Liberar memória utilizada pelo conteúdo antigo
  - Objeto receberá novo conteúdo

```
int main()
{
    String prog { "Prog" };
    String lang { "C++" };

    prog = lang;
}
```

```
prog.operator=(lang);
```

```
String & String::operator=(const String & s)
{
    ...

    delete [] str;              // libera memória
    tam = s.tam;                // copia tamanho
    str = new char[tam + 1];    // aloca memória
    strcpy(str, s.str);         // copia string

    ...
}
```

Operador de Atribuição

- Retornar uma referência
  - Ao objeto modificado (alvo da atribuição)
  - Permite atribuições encadeadas

```
int main()
{
    String lang { "C++" };
    String prog;
    String code;

    code = prog = lang;
}
```

```
code.operator=(prog.operator=(lang));
```

```
String & String::operator=(const String & s)
{
    delete [] str;              // libera memória
    tam = s.tam;                // copia tamanho
    str = new char[tam + 1];    // aloca memória
    strcpy(str, s.str);         // copia string

    return *this;               // retorna objeto
}
```

Operador de Atribuição

- Verificar atribuição para si mesmo
  - Pode retornar sem fazer nada

```
int main()
{
    String prog { "Prog" };
    String lang { "C++" };

    lang = lang;
}
```

```
lang.operator=(lang);
```

```
String & String::operator=(const String & s)
{
    if (this == &s)             // atribuído a si mesmo?
        return *this;           // retorna objeto

    delete [] str;              // libera memória
    tam = s.tam;                // copia tamanho
    str = new char[tam + 1];    // aloca memória
    strcpy(str, s.str);         // copia string

    return *this;               // retorna objeto
}
```

Observações

- Atribuição não cria objetos
  - Objeto já foi construído antes
  - Não incrementa número de instâncias (objs++)

```
class String
{
private:
   char * str;
   size_t tam;
   static int objs;

   ...
};
```

```
int main()
{
    String prog { "Prog" };
    String lang { "C++" };

    prog = lang;    // não cria objeto "prog",
                    // ele já foi construído
                    // antes da atribuição
}
```

Observações

- operator= só pode ser método de uma classe
  - Assim como outros operadores
  - Não podem ser função

```
class String
{
   ...                                  // atributos

public:
   String(const char s[]);              // construtor
   String(const String& s);             // construtor de cópia
   String();                            // construtor padrão
   ~String();                           // destrutor

   String & operator=(const String & s); // operador de atribuição
};
```

=	( )	[ ]	->

Operadores que só podem ser métodos

Observações

- É possível impedir o uso do operador de atribuição
  - Assim como no construtor de cópia
  - Funciona a partir do C++11

Operador de Atribuição não é gerado automaticamente pelo compilador

```
String & operator=(const String & s) = delete ;
```

```
prog = lang;
```

```
Error C2280: 'String & String::operator=(const String &)':
attempting to reference a deleted function
```

A Classe String

- Mais uma atualização:

```
class String
{
private:
   char * str;                          // ponteiro para string
   size_t tam;                          // tamanho da string
   static int objs;                     // número de instâncias

public:
   String(const char s[]);              // construtor
   String(const String& s);             // construtor de cópia
   String();                            // construtor padrão
   ~String();                           // destrutor

   String & operator=(const String & s);   // operador de atribuição

   friend ostream& operator<<(ostream& os, const String& s);
};
```

» String.h

A Classe String

```cpp
#include "String.h"
#include <cstring>

int String::objs = 0;

String::String(const char s[])
{
    tam = strlen(s);
    str = new char[tam + 1];
    strcpy(str, s);
    objs++;
}

String::String(const String& s)
{
    tam = s.tam;
    str = new char[tam + 1];
    strcpy(str, s.str);
    objs++;
}
```

```cpp
class String
{
private:
   char * str;
   size_t tam;
   static int objs;

public:
   String(const char s[]);
   String(const String& s);
   String();
   ~String();

   String & operator=(const String & s)
   friend ostream& operator<<
     (ostream& os, const String& s);
};
```

» String.cpp

A Classe String

```
String::String()
{
    tam = 0;
    str = new char[1] { "" };
    objs++;
}


String::~String()
{
    delete [] str;
    objs--;
}
```

```
class String
{
private:
   char * str;
   size_t tam;
   static int objs;

public:
   String(const char s[]);
   String(const String& s);
   String();
   ~String();

   String & operator=(const String & s)
   friend ostream& operator<<
     (ostream& os, const String& s);
};
```

» String.cpp

A Classe String

```
String & String::operator=(const String & s)
{
    if (this == &s)
        return *this;

    delete [] str;
    tam = s.tam;
    str = new char[tam + 1];
    strcpy(str, s.str);

    return *this;
}

ostream& operator<<
    (ostream& os, const String& s) {
    os << s.str;
    return os;
}
```

```
class String
{
private:
   char * str;
   size_t tam;
   static int objs;

public:
   String(const char s[]);
   String(const String& s);
   String();
   ~String();

   String & operator=(const String & s)
   friend ostream& operator<<
     (ostream& os, const String& s);
};
```

» String.cpp

A Classe String

```
void Valor(String s)
{
     cout << "Valor: " << s << "\n";
}

int main()
{
     String atual { "Artemis II retorna à Terra" };
     String tecno { "YouTube agora permite desativar Shorts" };

     Valor(atual);                                  // construtor de cópia
     cout << "Atual: " << atual << endl;

     String urgente = atual;                        // construtor de cópia
     cout << "Urgente: " << urgente << endl;

     urgente = tecno;                               // operador de atribuição
     cout << "Mais urgente: " << urgente << endl;
}
```

» Noticias.cpp

A Classe String

- Saída do programa:

```
Valor: Artemis II retorna à Terra
Atual: Artemis II retorna à Terra

Urgente: Artemis II retorna à Terra
Mais urgente: YouTube agora permite desativar Shorts
```

  - Construtor de cópia e Operador de atribuição foram usados
    - Alocações e cópias funcionam corretamente

Resumo

- A alocação dinâmica de memória requer:
  - Construtor: alocar memória
  - Destrutor: liberar memória
  - Construtor de cópia: copiar memória alocada
  - Operador de atribuição: copiar memória alocada
- O operador de atribuição é usado:
  - Em atribuições
  - Não em inicializações

Apoie

SEJA MEMBRO
Pagamento recorrente. Cancele quando quiser. O criador de conteúdo pode mudar os benefícios.
Selos de fidelidade ao lado do seu nome em comentários e no chat ao vivo
Emoji personalizado para usar em comentários e chat ao vivo
YouTube

Ciência da Computação

Programação Orientada a Objetos

APRESENTAÇÃO DO CURSO

em C++

INSCREVA-SE

Ciência da Computação

Programação Orientada a Objetos

APRESENTAÇÃO DO CURSO

em **C++**

INSCREVA-SE